Convex Optimization in `nabia`

L-BFGS, the augmented Lagrangian method and SQP: theory, algorithms and implementation

nabia documentation

October 2026

Abstract

This document describes the algorithms in the `nabia` module `optimization/convex`: the limited-memory BFGS method (`LBFGS`) for smooth unconstrained problems, and two solvers for smooth problems with linear equality, inequality and box constraints: the augmented Lagrangian method, also called the method of multipliers (`AugmentedLagrangian`), and sequential quadratic programming (`SQP`). We derive the augmented Lagrangian for equalities and inequalities and the multiplier updates. We then discuss convergence in the convex case and the practical safeguards (penalty update, inexact inner solves, stopping tests), and present L-BFGS as both a standalone solver and the inner solver. Finally we derive SQP: its quadratic subproblem, the damped BFGS Hessian approximation and the ℓ_1 merit function, and compare when each constrained solver is preferable.

Contents

1 Module overview **2**

2 Problem statement **2**

3 From penalties to augmented Lagrangians **3**
3.1 The quadratic penalty and its drawback 3
3.2 Shifting the penalty: equality constraints 3
3.3 Inequality constraints 3

4 Convergence in the convex case **4**

5 Practical algorithm **5**

6 The inner solver: L-BFGS **5**

7 Sequential quadratic programming **6**
7.1 The QP subproblem 7
7.2 Damped BFGS Hessian approximation 7
7.3 Merit function and line search 7
7.4 Choosing a constrained solver 8

8 Box constraints **9**

9 Using the implementation **9**
9.1 Augmented Lagrangian 9
9.2 Sequential quadratic programming 10
9.3 L-BFGS on its own 10
9.4 Scaling advice 10

1 Module overview

Type	Role
`IOptimizer<T, R>`	Base interface: owns the parameters `T` through a `std::unique_ptr`, declares `optimize(initial_guess)`, which returns the solution, and `result()`, which returns the solver's result object `R`.
`LBFGS`	`IOptimizer<LBFGSParams, LBFGSResult>`; unconstrained smooth minimization (Section 6).
`IConvexOptimization<R>`	`IOptimizer<ConvexOptimizationParams, R>`; interface for constrained convex solvers.
`AugmentedLagrangian`	`IConvexOptimization<AugmentedLagrangianResult>`; linearly constrained convex minimization, using `LBFGS` for its subproblems.
`SQP`	`IConvexOptimization<SQPResult>`; linearly constrained smooth minimization with few evaluations of f (Section 7); solves its QP subproblems with `AugmentedLagrangian`.
`LinearConstraint`	$A_{\text{eq}}x = b_{\text{eq}}$ and $A_{\text{in}}x \leq b_{\text{in}}$ as Eigen matrices and vectors.
`BoxConstraints`	Bounds $\ell \leq x \leq u$ (Section 8).
`Constraints`	`std::variant<BoxConstraints, LinearConstraint>`; held as `std::optional` in `ConvexOptimizationParams`, where empty means unconstrained. `to_linear_constraint` converts any of them to matrix form.

Table 1: Types in `nabia/optimization`.

All solvers follow the same pattern: construct with a parameter object and the objective (and gradient), then call `optimize(std::optional<Eigen::VectorXd> initial_guess)`. It returns the solution and starts from zero when no guess is given. Diagnostics such as multipliers, residuals and iteration counts are in `result()`. The same solver can be run again from a different starting point. Invalid input (mismatched dimensions, inconsistent constraints, nonpositive tolerances) raises `std::invalid_argument`.

2 Problem statement

We consider

$$\min_{x \in \mathbb{R}^n} \; f(x) \quad \text{s.t.} \quad h(x) \coloneqq A_{\text{eq}}x - b_{\text{eq}} = 0, \qquad g(x) \coloneqq A_{\text{in}}x - b_{\text{in}} \leq 0, \tag{1}$$

with $f : \mathbb{R}^n \to \mathbb{R}$ convex and continuously differentiable, $A_{\text{eq}} \in \mathbb{R}^{p \times n}$, $A_{\text{in}} \in \mathbb{R}^{m \times n}$. Box constraints $\ell \leq x \leq u$ are a special case of the inequalities (Section 8).

Lagrangian and KKT conditions. With multipliers $\lambda \in \mathbb{R}^p$ for the equalities and $\mu \in \mathbb{R}^m$ for the inequalities, the Lagrangian is

$$L(x, \lambda, \mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x).$$

Because the constraints are linear, no further constraint qualification is needed: a point $x^\star$ solves (1) if and only if there exist $(\lambda^\star, \mu^\star)$ satisfying the Karush–Kuhn–Tucker conditions

$$\nabla f(x^\star) + A_{\mathrm{eq}}^\top \lambda^\star + A_{\mathrm{in}}^\top \mu^\star = 0 \qquad \text{(stationarity)} \tag{2}$$

$$A_{\mathrm{eq}} x^\star = b_{\mathrm{eq}}, \quad A_{\mathrm{in}} x^\star \le b_{\mathrm{in}} \qquad \text{(primal feasibility)} \tag{3}$$

$$\mu^\star \ge 0 \qquad \text{(dual feasibility)} \tag{4}$$

$$\mu_i^\star \, g_i(x^\star) = 0, \; i = 1, \dots, m \qquad \text{(complementarity)} \tag{5}$$

The multipliers have a sensitivity interpretation: $\lambda_i^\star$ and $\mu_i^\star$ are the rates of change of the optimal value with respect to the right-hand sides b_i (with opposite sign).

3 From penalties to augmented Lagrangians

3.1 The quadratic penalty and its drawback

The simplest way to remove the equality constraints is the quadratic penalty

$$Q_\rho(x) = f(x) + \frac{\rho}{2} \|h(x)\|^2.$$

Its minimizer x_ρ satisfies $\nabla f(x_\rho) + A_{\mathrm{eq}}^\top (\rho \, h(x_\rho)) = 0$. Comparing with (2), $\rho \, h(x_\rho)$ plays the role of the multiplier. Since $\lambda^\star$ is generally nonzero, $h(x_\rho) \approx \lambda^\star / \rho$ vanishes only as $\rho \to \infty$. The Hessian of Q_ρ contains the term $\rho \, A_{\mathrm{eq}}^\top A_{\mathrm{eq}}$, so its condition number grows like ρ and the subproblems become increasingly hard to solve accurately.

3.2 Shifting the penalty: equality constraints

The augmented Lagrangian adds the penalty to the Lagrangian instead of to f:

$$L_\rho(x, \lambda) = f(x) + \lambda^\top h(x) + \frac{\rho}{2} \|h(x)\|^2, \qquad \nabla_x L_\rho = \nabla f(x) + A_{\mathrm{eq}}^\top (\lambda + \rho \, h(x)). \tag{6}$$

If x^+ minimizes $L_\rho(\cdot, \lambda)$, then $\nabla f(x^+) + A_{\mathrm{eq}}^\top (\lambda + \rho h(x^+)) = 0$. This is exactly the stationarity condition (2) with the multiplier estimate

$$\lambda^+ = \lambda + \rho \, h(x^+). \tag{7}$$

Now $h(x^+) = (\lambda^+ - \lambda)/\rho$, so feasibility only requires the multiplier estimates to converge, not $\rho \to \infty$. If $\lambda = \lambda^\star$, then $x^\star$ already minimizes $L_\rho(\cdot, \lambda^\star)$ for every $\rho > 0$: the penalty is *exact* at the right multiplier.

3.3 Inequality constraints

Write each inequality with a slack, $g_i(x) + s_i = 0$, $s_i \ge 0$, and apply (6) to the equality $g(x) + s = 0$:

$$\min_{x, \, s \ge 0} \; f(x) + \mu^\top (g(x) + s) + \frac{\rho}{2} \|g(x) + s\|^2.$$

The minimization over s separates by component and has a closed form. For fixed x, the unconstrained minimizer of $\mu_i (g_i + s_i) + \frac{\rho}{2}(g_i + s_i)^2$ is $s_i = -g_i - \mu_i/\rho$; projecting onto $s_i \ge 0$ gives $s_i^\star = \max\{0, \, -g_i - \mu_i/\rho\}$. Substituting back:

- if $\mu_i + \rho g_i \le 0$ (constraint comfortably inactive), then $g_i + s_i^\star = -\mu_i/\rho$ and the term equals $-\mu_i^2/(2\rho)$;
- if $\mu_i + \rho g_i > 0$, then $s_i^\star = 0$ and the term equals $\mu_i g_i + \frac{\rho}{2} g_i^2 = \big((\mu_i + \rho g_i)^2 - \mu_i^2\big)/(2\rho)$.

Both cases combine into the Powell–Hestenes–Rockafellar (PHR) augmented Lagrangian

$$\boxed{L_\rho(x,\lambda,\mu) = f(x) + \lambda^\top h(x) + \frac{\rho}{2}\|h(x)\|^2 + \frac{1}{2\rho}\sum_{i=1}^{m}\Big(\,[\mu_i + \rho\, g_i(x)]_+^2 - \mu_i^2\Big)} \tag{8}$$

with $[t]_+ = \max\{0, t\}$, and gradient

$$\nabla_x L_\rho(x,\lambda,\mu) = \nabla f(x) + A_{\text{eq}}^\top(\lambda + \rho\, h(x)) + A_{\text{in}}^\top\,[\mu + \rho\, g(x)]_+\,. \tag{9}$$

Applying (7) to $g + s^\star$ gives the inequality update

$$\mu^+ = \mu + \rho(g(x^+) + s^\star) = \Big[\mu + \rho\, g(x^+)\Big]_+\,, \tag{10}$$

which keeps $\mu^+ \geq 0$ automatically.

Properties of (8):

- $L_\rho(\cdot,\lambda,\mu)$ is convex whenever f is (the extra terms are convex functions of affine maps), and continuously differentiable.
- It is *not* twice differentiable where $\mu_i + \rho g_i(x) = 0$. This is why a quasi-Newton inner solver (L-BFGS) is a good fit: it only uses gradients.
- With the updated multipliers, the gradient (9) at x^+ is exactly the stationarity residual of the KKT system: $\nabla_x L_\rho(x^+,\lambda,\mu) = \nabla f(x^+) + A_{\text{eq}}^\top\lambda^+ + A_{\text{in}}^\top\mu^+$. The inner solver's gradient norm therefore measures KKT stationarity directly.

4 Convergence in the convex case

Dual interpretation. Let $d(\lambda,\mu) = \inf_x L(x,\lambda,\mu)$ be the (concave) dual function. Rockafellar [4] showed that one step of the method of multipliers, minimizing (8) and applying (7)–(10), is exactly one step of the *proximal point algorithm* applied to the dual:

$$(\lambda^+,\mu^+) = \operatorname*{arg\,max}_{\lambda,\,\mu\geq 0}\ \Big\{d(\lambda,\mu) - \frac{1}{2\rho}\|(\lambda,\mu) - (\lambda_k,\mu_k)\|^2\Big\}.$$

Equivalently, it is a gradient ascent step of length ρ on the smoothed dual, whose gradient is the constraint residual. Consequences for convex problems:

- For *any* fixed $\rho > 0$, if (1) has a KKT point, the multipliers converge to a dual solution and every limit point of x_k is primal optimal. Unlike the pure penalty method, ρ does not have to go to infinity.
- This remains true when the subproblems are solved inexactly, provided the errors are summable (e.g. inner tolerances $\varepsilon_k \to 0$ geometrically).
- The rate is linear, and the linear constant improves as ρ grows; with $\rho_k \to \infty$ it is superlinear. Larger ρ makes outer iterations more effective but inner problems more ill-conditioned; the practical rule of Section 5 balances both.

For nonconvex f the method still works locally (under second-order sufficiency, for ρ above a threshold), but `nabia` targets the convex case.

5 Practical algorithm

Measuring infeasibility. For the inequalities, $g_i \le 0$ alone is not enough: complementarity (5) must hold too. We use the measure of Birgin and Martínez [9]

$$V_k = \max\Big\{\|h(x_k)\|_\infty,\ \|\max\{g(x_k),\, -\mu_k/\rho_k\}\|_\infty\Big\}, \tag{11}$$

evaluated *before* the multiplier update. The i-th inequality term is zero iff $g_i \le 0$ and either $g_i = 0$ or $\mu_i = 0$. Moreover, $\max\{g, -\mu/\rho\} = (\mu^+ - \mu)/\rho$, so V_k also measures how much the multipliers still move.

Penalty update. If the violation did not decrease enough, $V_k > \tau\, V_{k-1}$ (default $\tau = 0.25$), the multipliers alone are not making fast enough progress and the penalty is increased, $\rho_{k+1} = \min\{\gamma\rho_k, \rho_{\max}\}$ (default $\gamma = 10$). Otherwise ρ is kept.

Inexact inner solves. Early outer iterations use inaccurate multipliers, so solving their subproblems to high accuracy is wasted work. The inner tolerance starts at $\varepsilon_0 = \max\{\texttt{tol},\, 10^{-1}\}$ and shrinks as $\varepsilon_{k+1} = \max\{\texttt{tol},\, 0.1\,\varepsilon_k\}$. Each subproblem is warm started at the previous x_k.

Stopping test. The method stops when both KKT residuals are below the user tolerance:

$$\|\nabla_x L_{\rho_k}(x_{k+1}, \lambda_k, \mu_k)\|_\infty \le \texttt{tol} \quad \text{and} \quad V_k \le \texttt{tol}.$$

By the identity at the end of Section 2, the first condition is stationarity (2) for the updated multipliers. The multipliers are nonnegative by construction, and V_k covers feasibility and complementarity. So the test certifies an approximate KKT point.

Algorithm 1 Augmented Lagrangian (PHR) with L-BFGS inner solver

Require: x_0, tolerance `tol`, $\rho_0 > 0$, $\gamma > 1$, $\tau \in (0,1)$, $\rho_{\max}$
$\lambda_0 \leftarrow 0$, $\mu_0 \leftarrow 0$, $\varepsilon_0 \leftarrow \max\{\texttt{tol}, 10^{-1}\}$, $V_{-1} \leftarrow \infty$
for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**
  $x_{k+1} \leftarrow \arg\min_x L_{\rho_k}(x, \lambda_k, \mu_k)$ from x_k, up to $\|\nabla_x L_{\rho_k}\|_\infty \le \varepsilon_k$ ▷ Alg. 2
  $V_k \leftarrow \max\{\|h(x_{k+1})\|_\infty,\ \|\max\{g(x_{k+1}), -\mu_k/\rho_k\}\|_\infty\}$
  $\lambda_{k+1} \leftarrow \lambda_k + \rho_k\, h(x_{k+1})$
  $\mu_{k+1} \leftarrow [\mu_k + \rho_k\, g(x_{k+1})]_+$
  if $\|\nabla_x L_{\rho_k}(x_{k+1}, \lambda_k, \mu_k)\|_\infty \le \texttt{tol}$ **and** $V_k \le \texttt{tol}$ **then**
    return $(x_{k+1}, \lambda_{k+1}, \mu_{k+1})$ ▷ approximate KKT point
  end if
  if $V_k > \tau V_{k-1}$ **then**
    $\rho_{k+1} \leftarrow \min\{\gamma\rho_k, \rho_{\max}\}$
  else
    $\rho_{k+1} \leftarrow \rho_k$
  end if
  $\varepsilon_{k+1} \leftarrow \max\{\texttt{tol},\, 0.1\,\varepsilon_k\}$
end for

When there are no constraints, (8) reduces to f, and the implementation solves a single subproblem directly to the tolerance `tol`.

6 The inner solver: L-BFGS

Each subproblem $\min_x L_\rho(x, \lambda, \mu)$ is smooth and unconstrained. `nabia` solves it with limited-memory BFGS [6, 7], which is also exposed as the standalone optimizer `LBFGS`.

Quasi-Newton direction. BFGS keeps an approximation H_k of the inverse Hessian and moves along $d_k = -H_k \nabla f(x_k)$. With $s_k = x_{k+1} - x_k$, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ and $\rho_k = 1/(y_k^\top s_k)$, the update is

$$H_{k+1} = (I - \rho_k s_k y_k^\top)\, H_k\, (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top .$$

L-BFGS never forms H_k. It stores the last M pairs (s_j, y_j) and computes $H_k \nabla f$ with the two-loop recursion (Algorithm 2) in $O(Mn)$ time and memory. The initial matrix is $H_k^0 = \gamma_k I$ with $\gamma_k = s_{k-1}^\top y_{k-1} / y_{k-1}^\top y_{k-1}$, which gives the step a meaningful scale.

Safeguards.

- A pair is stored only if $s^\top y > 10^{-12} \|s\| \|y\|$. This keeps H_k positive definite and d_k a descent direction. For convex f the curvature condition $s^\top y \geq 0$ always holds.
- If d_k is not a descent direction (for example due to round-off), the memory is cleared and a normalized steepest-descent step is taken.
- The step length t is chosen by backtracking ($t \leftarrow 0.5t$) from $t = 1$ until the Armijo condition $f(x_k + td_k) \leq f(x_k) + c_1 t\, \nabla f(x_k)^\top d_k$ holds (default $c_1 = 10^{-4}$). Non-finite trial values are rejected.
- *Approximate Wolfe acceptance.* Close to a minimizer, the decrease $c_1 t\, |\nabla f^\top d|$ that Armijo demands falls below the rounding error in f. The test then stalls even though the gradient, which is still accurate, would allow progress. Following Hager and Zhang [12], a step is also accepted when f is flat to within rounding, $f(x_k + td_k) \leq f(x_k) + \epsilon_f |f(x_k)|$, and the directional derivative satisfies $\sigma\, \nabla f(x_k)^\top d_k \leq \nabla f(x_k + td_k)^\top d_k \leq (2c_1 - 1)\, \nabla f(x_k)^\top d_k$ (defaults $\epsilon_f = 10^{-10}$, $\sigma = 0.9$). Without this rule the risk budgeting solver, for example, stalls at an accuracy of about $3 \cdot 10^{-8}$ on problems with hundreds of assets.

Algorithm 2 L-BFGS with Armijo backtracking

Require: x_0, memory M, tolerance ε

for $k = 0, 1, \ldots$ **do**
  if $\|\nabla f(x_k)\|_\infty \leq \varepsilon$ **then return** x_k
  end if
  $q \leftarrow \nabla f(x_k)$
  for $j = k-1, \ldots, k-M$ **do** ▷ newest to oldest
    $\alpha_j \leftarrow \rho_j s_j^\top q$; $q \leftarrow q - \alpha_j y_j$
  end for
  $r \leftarrow \gamma_k q$
  for $j = k-M, \ldots, k-1$ **do** ▷ oldest to newest
    $\beta \leftarrow \rho_j y_j^\top r$; $r \leftarrow r + (\alpha_j - \beta) s_j$
  end for
  $d_k \leftarrow -r$; **if** $\nabla f(x_k)^\top d_k \geq 0$ **then** clear memory, $d_k \leftarrow -\nabla f(x_k) / \max\{1, \|\nabla f(x_k)\|\}$
  $t \leftarrow 1$; **while** $f(x_k + td_k) > f(x_k) + c_1 t \nabla f(x_k)^\top d_k$ **do** $t \leftarrow t/2$
  $x_{k+1} \leftarrow x_k + td_k$; store (s_k, y_k) if $s_k^\top y_k > 0$, dropping the oldest pair beyond M
end for

7 Sequential quadratic programming

The augmented Lagrangian method needs many evaluations of f: each outer iteration is a full L-BFGS run. Sequential quadratic programming (SQP) instead builds a quadratic model of the problem at every iterate. It solves that model, which costs no evaluations of f, and takes one

step. When f is expensive and n is moderate (SQP stores a dense $n \times n$ matrix), this is usually far cheaper overall.

7.1 The QP subproblem

SQP is Newton's method applied to the KKT conditions. For (1), linearizing the KKT system at the current iterate x_k gives the optimality conditions of the quadratic program

$$\min_{d \in \mathbb{R}^n} \ \nabla f(x_k)^\top d + \tfrac{1}{2}\, d^\top B_k d \quad \text{s.t.} \quad A_{\text{eq}}(x_k + d) = b_{\text{eq}}, \qquad A_{\text{in}}(x_k + d) \le b_{\text{in}}, \tag{12}$$

with $B_k \approx \nabla^2_{xx} L(x_k, \lambda, \mu)$. Because the constraints are linear, two simplifications follow:

- The linearized constraints are the constraints themselves. The QP is feasible whenever (1) is, and a full step d lands on a feasible point, even from an infeasible x_k.
- The constraints have zero curvature, so $\nabla^2_{xx} L = \nabla^2 f$. B_k only has to approximate the Hessian of f, and the multipliers do not enter it.

The KKT conditions of (12) are

$$\nabla f(x_k) + B_k d + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu = 0, \quad \mu \ge 0, \quad \mu_i (g_i(x_k) + (A_{\text{in}} d)_i) = 0,$$

plus feasibility. The QP multipliers (λ, μ) are therefore estimates of the multipliers of (1). If $d = 0$ solves the QP, then x_k is a KKT point of the original problem.

7.2 Damped BFGS Hessian approximation

With $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, the BFGS update keeps B_k positive definite only if $s_k^\top y_k > 0$. That always holds for strictly convex f, but can fail otherwise. Powell's damping [10] replaces y_k by

$$r_k = \theta_k y_k + (1 - \theta_k) B_k s_k, \qquad \theta_k = \begin{cases} 1 & \text{if } s_k^\top y_k \ge 0.2\, s_k^\top B_k s_k, \\ \dfrac{0.8\, s_k^\top B_k s_k}{s_k^\top B_k s_k - s_k^\top y_k} & \text{otherwise,} \end{cases}$$

which guarantees $s_k^\top r_k \ge 0.2\, s_k^\top B_k s_k > 0$, and updates

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{r_k r_k^\top}{s_k^\top r_k}.$$

$B_0 = I$ is rescaled to $(y_0^\top y_0 / s_0^\top y_0)\, I$ after the first step, as in L-BFGS. Positive definiteness makes every QP strictly convex, so its solution is unique.

7.3 Merit function and line search

The full step can increase f or fail to make progress, so the step length t is chosen on the ℓ_1 merit function [11]

$$\phi_\nu(x) = f(x) + \nu\, V_1(x), \qquad V_1(x) = \|h(x)\|_1 + \|[g(x)]_+\|_1.$$

For ν larger than every multiplier, ϕ_ν is an *exact* penalty: its minimizers are the solutions of (1).

The QP step is a descent direction. Since the constraints are linear and d satisfies them, $V_1(x_k + td) \le (1-t)V_1(x_k)$ for $t \in [0,1]$. The directional derivative is therefore $D\phi_\nu(x_k; d) \le \nabla f^\top d - \nu V_1(x_k)$. From the QP optimality conditions, $A_{\text{eq}} d = -h$, and complementarity gives $\mu^\top A_{\text{in}} d = -\mu^\top g$. Hence

$$\nabla f^\top d = -d^\top B_k d + \lambda^\top h + \mu^\top g \le -d^\top B_k d + \|(\lambda, \mu)\|_\infty V_1(x_k),$$

using $\mu \ge 0$ and $\mu^\top g \le \mu^\top [g]_+$. So

$$D\phi_\nu(x_k; d) \le -d^\top B_k d - (\nu - \|(\lambda, \mu)\|_\infty) V_1(x_k) < 0$$

whenever $d \neq 0$ and $\nu > \|(\lambda, \mu)\|_\infty$. The implementation keeps $\nu \leftarrow \max\{\nu, \|(\lambda, \mu)\|_\infty + \delta\}$ with $\delta = 10^{-3}$ and backtracks from $t = 1$ until the Armijo condition $\phi_\nu(x_k + td) \le \phi_\nu(x_k) + c_1 t\,(\nabla f^\top d - \nu V_1(x_k))$ holds.

Convergence. Every accepted step decreases the merit function, which makes the method robust from poor (even infeasible) starting points. Under standard assumptions (bounded B_k and multipliers) the iterates approach KKT points. Near a solution that satisfies the standard second-order conditions, the full step $t = 1$ is eventually accepted. Convergence is then superlinear with a BFGS approximation, and quadratic with the exact Hessian [8].

Stopping test. At each iterate the QP multipliers give the KKT residuals

$$\|\nabla f(x_k) + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu\|_\infty, \qquad \max\{\|h(x_k)\|_\infty, \|[g(x_k)]_+\|_\infty\}, \qquad \|\mu \odot g(x_k)\|_\infty,$$

that is, stationarity (which equals $\|B_k d\|_\infty$), feasibility and complementarity. SQP stops when all three are below `tolerance`.

Solving the QP. Without constraints, (12) is solved directly by a Cholesky factorization, $d = -B_k^{-1} \nabla f$. Otherwise it is a strictly convex QP with linear constraints, which `AugmentedLagrangian` solves to a tolerance of 10^{-10}. This costs no evaluations of f, and also gives the multipliers.

Algorithm 3 SQP with damped BFGS and ℓ_1 merit line search

Require: x_0 (may be infeasible), tolerance `tol`, $\nu_0 > 0$

$B \leftarrow I$
for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**
  $(d, \lambda, \mu) \leftarrow$ solution and multipliers of the QP (12) at x_k
  if stationarity, feasibility and complementarity at x_k are $\le$ `tol` **then return** (x_k, λ, μ)
  end if
  $\nu \leftarrow \max\{\nu, \|(\lambda, \mu)\|_\infty + \delta\}$
  $t \leftarrow 1$; **while** $\phi_\nu(x_k + td) > \phi_\nu(x_k) + c_1 t(\nabla f^\top d - \nu V_1(x_k))$ **do** $t \leftarrow t/2$
  if the line search fails **then** reset $B \leftarrow I$ and retry (stop if B already was I)
  $x_{k+1} \leftarrow x_k + td$; $s \leftarrow x_{k+1} - x_k$; $y \leftarrow \nabla f(x_{k+1}) - \nabla f(x_k)$
  $B \leftarrow$ damped BFGS update of B with (s, y) ▷ rescale B first, at $k = 0$
end for

7.4 Choosing a constrained solver

On the 5-variable simplex problem of the test suite, with a non-quadratic f, SQP converges in 12 iterations using 15 evaluations of f. The augmented Lagrangian needs 89 L-BFGS iterations to reach the same point.

	Augmented Lagrangian	SQP
Memory	$O(Mn)$ (L-BFGS pairs)	$O(n^2)$ (dense B)
Evaluations of f	many (one L-BFGS run per outer step)	few (one QP per step)
Work per step	cheap	a QP solve
Local rate	linear	superlinear
Best for	large n, cheap f	moderate n, expensive f

Table 2: Trade-offs between the two constrained solvers.

8 Box constraints

`BoxConstraints` $\ell \le x \le u$ are turned into rows of $A_{\text{in}}x \le b_{\text{in}}$. Every finite u_i gives the row $e_i^\top x \le u_i$, and every finite ℓ_i gives $-e_i^\top x \le -\ell_i$. Infinite bounds produce no row, and an empty bound vector means that side is unbounded. In `result().mu_ineq` the upper-bound multipliers come first (in index order), followed by the lower-bound multipliers. Bounds with $\ell_i > u_i$ are rejected with `std::invalid_argument`. The conversion is the public function `to_linear_constraint` (`Constraints.h`), which both constrained solvers use, so their multiplier vectors follow the same row order.

9 Using the implementation

9.1 Augmented Lagrangian

```cpp
#include <nabia/optimization/convex/AugmentedLagrangian.h>

// min |x - c|^2 s.t. sum(x) = 1, x >= 0 (projection onto the simplex)
Eigen::VectorXd c(3); c << 1.0, -1.0, 0.5;

LinearConstraint lc;
lc.A_eq = Eigen::MatrixXd::Ones(1, 3); lc.b_eq = Eigen::VectorXd::Ones(1);
lc.A_ineq = -Eigen::MatrixXd::Identity(3, 3); lc.b_ineq = Eigen::VectorXd::Zero(3);


auto params = std::make_unique<ConvexOptimizationParams>(
    ConvexOptimizationParams{3, /*tolerance*/ 1e-8, /*max_iter*/ 100, lc});

AugmentedLagrangian solver(
    std::move(params),
    [&](const Eigen::VectorXd& x) { return (x - c).squaredNorm(); },
    [&](const Eigen::VectorXd& x) -> Eigen::VectorXd { return 2.0 * (x - c); });

Eigen::VectorXd x = solver.optimize(std::nullopt); // x = (0.75, 0, 0.25)
const auto& r = solver.result(); // r.lambda_eq = (0.5), r.mu_ineq = (0, 2.5, 0)
```

The KKT conditions can be checked by hand: $2(x - c) + \lambda\mathbf{1} - \mu = 0$ gives $\lambda = 0.5$ from the first component (where $\mu_1 = 0$) and $\mu_2 = 2(0 + 1) + 0.5 = 2.5$ for the active bound $x_2 = 0$.

The result type `AugmentedLagrangianResult` reports `x`, `lambda_eq`, `mu_ineq`, `objective`, `constraint_violation` (V_k), `stationarity`, the final `penalty`, the outer and total inner iteration counts, and `converged`.

Option	Default	Meaning
`initial_penalty`	10	ρ_0
`penalty_growth`	10	γ, factor applied to ρ when progress stalls
`max_penalty`	10^{10}	$\rho_{\max}$
`required_decrease`	0.25	τ in the test $V_k > \tau V_{k-1}$
`initial_inner_tolerance`	10^{-1}	ε_0 (never below `tolerance`)
`max_inner_iter`	1000	L-BFGS iterations per subproblem
`lbfgs_memory`	10	M, number of stored pairs

Table 3: Fields of `AugmentedLagrangianOptions`.

9.2 Sequential quadratic programming

`SQP` has the same constructor and `optimize` signature as `AugmentedLagrangian`, so switching between them is a one-word change:

```
#include <nabia/optimization/convex/SQP.h>

SQP solver(std::move(params), f, grad_f);
Eigen::VectorXd x = solver.optimize(x0); // x0 may be infeasible
const auto& r = solver.result(); // r.lambda_eq, r.mu_ineq, r.evaluations, ...
```

`SQPOptions` holds `qp_tolerance` (10^{-10}), `qp_max_iter` (200), the line-search constants `merit_c1` (10^{-4}), `backtrack` (0.5) and `max_line_search` (40), and the merit penalty settings `initial_merit_penalty` ($\nu_0 = 1$) and `merit_penalty_margin` ($\delta = 10^{-3}$). `SQPResult` reports the solution, the multipliers, the three KKT residuals, and the iteration and evaluation counts.

9.3 L-BFGS on its own

```
#include <nabia/optimization/convex/LBFGS.h>

// Rosenbrock, using a single callback for value and gradient.
auto fg = [](const Eigen::VectorXd& z, Eigen::VectorXd& grad) {
    const double x = z(0), y = z(1);
    grad(0) = -2.0 * (1.0 - x) - 400.0 * x * (y - x * x);
    grad(1) = 200.0 * (y - x * x);
    return (1.0 - x) * (1.0 - x) + 100.0 * (y - x * x) * (y - x * x);
};
Eigen::VectorXd x0(2); x0 << -1.2, 1.0;

LBFGS solver(std::make_unique<LBFGSParams>(LBFGSParams{2}), fg);
Eigen::VectorXd x = solver.optimize(x0); // x = (1, 1)
if (!solver.result().converged()) { /* inspect solver.result().status */ }
```

`LBFGSParams` holds `dim`, `tolerance` (default 10^{-8}), `max_iter` (1000), `memory` (10) and the line-search constants `armijo_c1`, `backtrack`, `max_line_search`, `flat_tolerance` (ϵ_f) and `wolfe_c2` (σ). `LBFGSResult::status` is `Converged`, `MaxIterations` or `LineSearchFailed`.

9.4 Scaling advice

The method is invariant to neither the scaling of f nor that of the constraint rows. If the rows of $A_{\text{eq}}, A_{\text{in}}$ have very different norms, or f takes values far from $O(1)$, normalize them first, since $\rho_0 = 10$ and the tolerances assume a well-scaled problem. If the solver ends with `converged == false` and a large `penalty`, the problem may be infeasible or badly scaled.

References

[1] M. R. Hestenes, Multiplier and gradient methods, *J. Optim. Theory Appl.* 4 (1969) 303–320.

[2] M. J. D. Powell, A method for nonlinear constraints in minimization problems, in R. Fletcher (ed.), *Optimization*, Academic Press, 1969, 283–298.

[3] R. T. Rockafellar, A dual approach to solving nonlinear programming problems by unconstrained optimization, *Math. Programming* 5 (1973) 354–373.

[4] R. T. Rockafellar, Augmented Lagrangians and applications of the proximal point algorithm in convex programming, *Math. Oper. Res.* 1 (1976) 97–116.

[5] D. P. Bertsekas, *Constrained Optimization and Lagrange Multiplier Methods*, Academic Press, 1982.

[6] J. Nocedal, Updating quasi-Newton matrices with limited storage, *Math. Comp.* 35 (1980) 773–782.

[7] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, *Math. Programming* 45 (1989) 503–528.

[8] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006, Chapters 7 and 17.

[9] E. G. Birgin and J. M. Martínez, *Practical Augmented Lagrangian Methods for Constrained Optimization*, SIAM, 2014.

[10] M. J. D. Powell, A fast algorithm for nonlinearly constrained optimization calculations, in G. A. Watson (ed.), *Numerical Analysis*, Lecture Notes in Mathematics 630, Springer, 1978, 144–157.

[11] S.-P. Han, A globally convergent method for nonlinear programming, *J. Optim. Theory Appl.* 22 (1977) 297–309.

[12] W. W. Hager and H. Zhang, A new conjugate gradient method with guaranteed descent and an efficient line search, *SIAM J. Optim.* 16 (2005) 170–192.